Reviewer Pack — Minimal Order of Tropical Abelianizations and DPLL Proof Len…

Entry 95a8225ac91c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 11:33:11 UTC

Minimal Order of Tropical Abelianizations and DPLL Proof Length

Entry ID: `95a8225ac91c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 11:33:11 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry × Category Theory **Field B** (complexity object): DPLL Proof Complexity

Statement:

> For every Boolean satisfiability instance φ, the minimal order of the tropical abelianization of its indicator polynomial modulo 2 is linearly correlated with the length of a DPLL proof for φ, such that ord_ab(φ) = Θ(l(φ)).

Rationale (proposer's reasoning):

> Tropical geometry has shown promise in capturing properties of computational structures through its algebraic operations. The tropical abelianization process, which maps a polynomial to an abelian group under tropical addition and multiplication, may reveal hidden structures within Boolean satisfiability instances that correlate with proof complexity. A constructive mapping from the instance to the field_A object can be achieved by computing the Grothendieck-Witt class of the indicator polynomial, a computable invariant in tropical geometry.

Taxonomy category: `TROPICAL_GEOMETRY_X_CATEGORY_THEORY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c5b8a7be493e30dd`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the Boolean satisfiability instances φ with size n ≤ 40, if the minimal order of tropical abelianization (ord_ab(φ)) is linearly correlated with the DPLL proof length (l(φ)), and this correlation is supported by a linear regression model on at least 30 random seeds with a correlation coefficient ≥ 0.7.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND category theory AND DPLL proof complexity - indicator polynomial tropical abelianization MOD 2 AND DPLL proof length - Boolean satisfiability instance AND minimal order tropical abelianization AND linear correlation with DPLL proof

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/math/0403015v1] Amoebas of algebraic varieties and tropical geometry - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism - [http://arxiv.org/abs/1204.6154v2] Local Tropicalization - [http://arxiv.org/abs/1103.5740v2] Generating and Searching Families of FFT Algorithms - [http://arxiv.org/abs/2206.00903v2] Satisfiability of Quantified Boolean Announcements - [http://arxiv.org/abs/math/0511713v2] Tropical Plane Geometric Constructions: a Transfer Technique in Tropical Geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def primes(n):
    sieve = [True] * (n+1)
    p = 2
    while (p * p <= n):
        if (sieve[p] == True):
            for i in range(p * p, n+1, p):
                sieve[i] = False
        p += 1
    return [p for p in range(2, n)]

def random_sat_instance(n: int) -> list:
    clauses = []
    variables = set()
    for _ in range(n):
        clause = []
        for _ in range(random.randint(1, n)):
```

```
            var = random.choice(list(variables)) if variables else f'v{random.randint(1, 2*n)}'
            variables.add(var)
            clause.append((var, random.choice([True, False])))
        clauses.append(clause)
    return clauses

def indicator_polynomial(cnf: list) -> dict:
    poly = {}
    for clause in cnf:
        term = 1
        for var, sign in clause:
            if sign:
                term *= (1 + int(var[1:]) if var.startswith('v') else -int(var[1:]))
            else:
                term *= (-1 + int(var[1:]) if var.startswith('v') else 1 - int(var[1:]))
        poly[tuple(sorted(clause))] = term
    return poly

def tropical_add(x, y):
    return max(x, y)

def tropical_multiply(x, y):
    return x + y

def tropical_abelianization(poly: dict) -> int:
    order = 0
    for clause in poly:
        term_order = sum(1 for var, _ in clause)
        if term_order > order:
            order = term_order
    return order

def dpll(cnf: list, assignment: dict = {}) -> bool:
    if not cnf:
        return True
    literal = next((l for l in cnf[0] if l[1]), None)
    if literal is None:
        return False
    var, sign = literal
    new_assignment = assignment.copy()
    new_assignment[var] = sign
    if dpll([c for c in cnf if not any(l == (var, s) for l, s in c)], new_assignment):
        return True
    new_assignment[var] = not sign
    return dpll([c for c in cnf if not any(l == (var, s) for l, s in c)], new_assignment)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 0
    metric_values = []
    conjecture_holds = True
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
```

```
            cnf = random_sat_instance(n)
            poly = indicator_polynomial(cnf)
            ord_ab = tropical_abelianization(poly)
            proof_length = len(dpll(cnf))
            metric_values.append((ord_ab, proof_length))
            instances_tested += 1
            if ord_ab != proof_length:
                conjecture_holds = False
                counterexample = f"n={n}, ord_ab={ord_ab}, proof_length={proof_length}"
                break

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": sum(x * y for x, y in metric_values) / (sum(x**2 for x, _ in metric_values
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else primes(30)
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = (sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(res
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f34172a.py", line 115, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f34172a.py", line 93, in run_trial
    proof_length = len(dpll(cnf))
                       ^^^^^^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f34172a.py", line 75, in dpll
    if dpll([c for c in cnf if not any(l == (var, s) for l, s in c)], new_assignment):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f34172a.py", line 75, in dpll
    if dpll([c for c in cnf if not any(l == (var, s) for l, s in c)], new_assignment):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f34172a.py", line 75, in dpll
    if dpll([c for c in cnf if not any(l == (var, s) for l, s in c)], new_assignment):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 994 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f34172a.py", line 69, in dpll
    literal = next((l for l in cnf[0] if l[1]), None)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means that it was unable to complete the required calculations and correlation analysis. | next: Re-run the test with a different implementation or error-handling strategy to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14859
2	preregistration	ollama_remote	glm4:latest	0	0	9580
3	novelty	ollama_remote	glm4:latest	0	0	13036
4	novelty	ollama_remote	glm4:latest	0	0	9864
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22498
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14012
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12628
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13221
9	judge	ollama_remote	glm4:latest	0	0	11913

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 121611 ms total latency. Provider mix: {‘ollama_remote’: 9}

(full prompt+response transcripts available in `research/audit/95a8225ac91c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/95a8225ac91c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/95a8225ac91c.tar.gz` (if generated)